

YADUVAR KANSAL

Id : 2023UEC1374

Grp : E3

Problem :

Simulate a telecommunication system with 4 channels where incoming calls follow a Poisson process with arrival rate $\lambda = 1$ and exponentially distributed service times with rate μ . For each arriving call, assign it to a channel if fewer than 4 channels are busy; otherwise, block the call. Run the simulation over a large number of calls, track active calls dynamically, and compute the blocking probability as the ratio of blocked calls to total calls. Also, calculate the theoretical blocking probability using the Erlang B formula with traffic intensity $A = \lambda / \mu$, and verify the result using steady-state probabilities of a birth-death Markov model where blocking occurs in state 4.

Soln :

```
%% =====
% Erlang B Simulation: 4-Channel Telecommunication System
% Methods: Monte Carlo Simulation | Erlang B | Markov Chain
% Parameters: C=4, lambda=1, mu=0.5 => A=2 Erlangs
% =====

clc; clear all; close all;

%% ---- 1. SYSTEM PARAMETERS -----
C = 4; % Number of channels (servers)
lambda = 1; % Arrival rate (calls/unit time) — Poisson process
mu = 0.5; % Service rate (calls/unit time) — mean service = 1/mu = 2
A = lambda / mu; % Traffic intensity in Erlangs
N = 10000; % Total number of calls to simulate

fprintf('=== System Parameters ===\n');
fprintf(' Channels (C) = %d\n', C);
fprintf(' Arrival rate (λ) = %.2f calls/unit time\n', lambda);
fprintf(' Service rate (μ) = %.2f calls/unit time\n', mu);
fprintf(' Traffic intensity A = %.2f Erlangs\n', A);
fprintf(' Calls simulated (N) = %d\n\n', N);

%% ---- 2. MONTE CARLO SIMULATION -----
% channel_end_times(i): time at which channel i becomes free
channel_end_times = zeros(1, C);
current_time = 0;
blocked = 0;
accepted = 0;
```

```

for i = 1:N
    % Inter-arrival time ~ Exp(lambda)
    inter_arrival = -log(rand) / lambda;
    current_time = current_time + inter_arrival;

    % Count channels still busy at current_time
    busy = sum(channel_end_times > current_time);

    if busy < C
        % At least one channel free — accept the call
        accepted = accepted + 1;
        service_time = -log(rand) / mu;      % Exp(mu)
        free_ch = find(channel_end_times <= current_time, 1);
        channel_end_times(free_ch) = current_time + service_time;
    else
        % All channels occupied — block (lost call)
        blocked = blocked + 1;
    end
end

Pb_sim = blocked / N;

fprintf('=== Simulation Results (N = %d) ===\n', N);
fprintf(' Accepted calls = %d\n', accepted);
fprintf(' Blocked calls = %d\n', blocked);
fprintf(' Simulated Pb = %.4f (%.2f%%)\n\n', Pb_sim, Pb_sim*100);

%% ---- 3. THEORETICAL: ERLANG B FORMULA -----
%
%      A^C / C!
% Pb = -----
%      Σ_{k=0}^C A^k / k!
%
numerator = (A^C) / factorial(C);
denominator = sum(arrayfun(@(k) A^k / factorial(k), 0:C));
Pb_theory = numerator / denominator;

fprintf('=== Erlang B Formula ===\n');
fprintf(' Theoretical Pb = %.4f (%.2f%%)\n\n', Pb_theory, Pb_theory*100);

%% ---- 4. MARKOV CHAIN (Birth-Death Model) -----
%

```

```

% Balance equations for M/M/C/C queue:
%  $\pi(n) = (A^n / n!) * \pi(0)$ ,  $n = 0, 1, \dots, C$ 
%
unnorm = arrayfun(@(n) A^n / factorial(n), 0:C);
pi = unnorm / sum(unnorm); % Normalize to get valid probabilities

fprintf('=== Markov Chain Steady-State Probabilities ===\n');
for n = 0:C
    marker = "";
    if n == C, marker = ' <-- Blocking State'; end
    fprintf(' P(state %d) = %.4f%s\n', n, pi(n+1), marker);
end

Pb_markov = pi(C+1); % Blocking = P(all C channels busy)
fprintf('\n Blocking Pb (Markov) = %.4f (%.2f%%)\n\n', Pb_markov, Pb_markov*100);

%% ---- 5. COMPARISON SUMMARY -----
fprintf('=== Comparison Summary ===\n');
fprintf(' %-20s Pb\n', 'Method');
fprintf(' %-20s %.4f\n', 'Simulation', Pb_sim);
fprintf(' %-20s %.4f\n', 'Erlang B Formula', Pb_theory);
fprintf(' %-20s %.4f\n', 'Markov Chain', Pb_markov);

%% ---- 6. PLOTS -----
figure(1);
set(gcf, 'Color', 'w', 'Position', [100 80 1100 750]);

% --- Subplot 1: Steady-State Probabilities (Markov) ---
subplot(2,2,1);
b = bar(0:C, pi, 'FaceColor', [0.2 0.55 0.85], 'EdgeColor', 'none');
hold on;
bar(C, pi(C+1), 'FaceColor', [0.85 0.2 0.2], 'EdgeColor', 'none');
xlabel('State n (number of busy channels)');
ylabel('Probability \pi(n)');
title('Steady-State Probabilities (Markov Chain)');
legend('P(state n)', 'Blocking State (n=C)', 'Location', 'northeast');
grid on; box off;
% Annotate each bar
for n = 0:C
    text(n, pi(n+1) + 0.005, sprintf('%.3f', pi(n+1)), ...
        'HorizontalAlignment', 'center', 'FontSize', 8);
end

```

```

% --- Subplot 2: Erlang B curve — Pb vs Traffic Intensity ---
subplot(2,2,2);
A_range = 0.1:0.1:10;
Pb_curve = zeros(size(A_range));
for idx = 1:length(A_range)
    a = A_range(idx);
    num = (a^C) / factorial(C);
    den = sum(arrayfun(@(k) a^k / factorial(k), 0:C));
    Pb_curve(idx) = num / den;
end
plot(A_range, Pb_curve, 'b-', 'LineWidth', 2);
hold on;
plot(A, Pb_theory, 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r');
xlabel('Traffic Intensity A (Erlangs)');
ylabel('Blocking Probability P_b');
title('Erlang B: P_b vs Traffic Intensity (C = 4)');
legend('Erlang B Curve', sprintf('Operating Point (A = %.1f)', A), ...
    'Location', 'northwest');
grid on; box off;

```

```

% --- Subplot 3: Pb vs Number of Channels (fixed A) ---
subplot(2,2,3);
C_range = 1:10;
Pb_vs_C = zeros(size(C_range));
for idx = 1:length(C_range)
    c = C_range(idx);
    num = (A^c) / factorial(c);
    den = sum(arrayfun(@(k) A^k / factorial(k), 0:c));
    Pb_vs_C(idx) = num / den;
end
stem(C_range, Pb_vs_C, 'filled', 'LineWidth', 1.5, 'Color', [0.2 0.5 0.8]);
hold on;
plot(C, Pb_theory, 'r*', 'MarkerSize', 14, 'LineWidth', 2);
xlabel('Number of Channels C');
ylabel('Blocking Probability P_b');
title(sprintf('P_b vs Channels (A = %.1f Erlangs)', A));
legend('P_b for each C', sprintf('Our System (C = %d)', C), ...
    'Location', 'northeast');
grid on; box off;

```

```

% --- Subplot 4: Comparison Bar Chart ---
subplot(2,2,4);
methods = {'Simulation', 'Erlang B', 'Markov Chain'};

```

```

values = [Pb_sim, Pb_theory, Pb_markov];
colors = [0.35 0.75 0.35; 0.25 0.55 0.85; 0.85 0.55 0.25];
for i = 1:3
    bar(i, values(i), 'FaceColor', colors(i,:), 'EdgeColor', 'none');
    hold on;
end
set(gca, 'XTick', 1:3, 'XTickLabel', methods);
ylabel('Blocking Probability P_b');
title('Comparison: Simulation vs Erlang B vs Markov Chain');
ylim([0, max(values) * 1.45]);
grid on; box off;
for i = 1:3
    text(i, values(i) + 0.002, sprintf('%.4f', values(i)), ...
        'HorizontalAlignment', 'center', 'FontWeight', 'bold', 'FontSize', 9);
end

sgtitle(sprintf('Erlang B Simulation | C = %d, \lambda = %.1f, \mu = %.2f, A = %.1f Erlangs', ...
    C, lambda, mu, A), 'FontSize', 13, 'FontWeight', 'bold');

```

Output Plot

Erlang B Simulation | $C = 4$, $\lambda = 1.0$, $\mu = 0.50$, $A = 2.0$ Erlangs

